GSMULTIAGENT · ARCHITECTURE BRIEF

制导系统设计中的多智能体架构：现状、对照与扩展

本项目不是「多个大模型互相聊天」的扁平多智能体系统，而是一套 **三层闭环 + Hermes 编排 + 工具隔离子智能体 + 规则先行的判断/反思** 架构。自然语言负责决策路径，MATLAB 蒙特卡洛负责物理真值，PPO/专家调参负责连续参数搜索。 本文说明现在怎么用、为什么这样用，以及后面还能往哪长。

入口 cli_agent.py | 编排 Hermes Agent | 优化 OptimizationWorkflow | 仿真 monte_carlo_single.m

1. 这个项目真正要协调的，不是「对话」，而是异质工作流

智能设计制导系统，一次任务至少要同时处理四类完全不同的工作：把自然语言工况变成 T/G/AP/R 实验设置； 必要时改 MATLAB 制导律结构；用蒙特卡洛给出命中率、SEP、PeakNy、PM、BW；再在 10 维连续参数空间里搜索。 这四件事的错误代价不一样：LLM 写错一句分析可以重来，MATLAB 语法错了仿真直接失败，指标判错会让 PPO 朝错误方向学。

离散结构决策

要不要改制导律（`MODIFY_LAW`），还是只调参（`TUNE_PARAMS`），或直接复用历史（`REUSE_HISTORY`）。这是符号/算法层。

连续参数搜索

驾驶仪 `w/ζ/τ` 与制导 `N_pn`、`ny_lim`。这是数值优化层，不适合让 LLM 每次猜一组数。

硬约束验收

五项指标必须全部过关。论文实验还要求可消融、可复现、可对齐 225 次 MC 预算。这是科学实验层。

因此架构的核心命题是：**把「会说话的模型」限制在决策与生成上，把「会算的引擎」放在仿真与优化上，用明确的角色边界防止幻觉进入验收环节。** 单智能体把这四类工作塞进同一次 ReAct 循环，短期能跑通演示，但无法支撑论文里的分层消融，也无法保证指标判定不被语言模型改写。

2. 当前架构全景：三层闭环，而不是对等辩论

运行时主轴是 `cli_agent.py` 驱动的外层循环（默认最多 5 轮）。每一轮先走 Layer 2 生成或复用脚本并仿真， 再把脚本与初始指标交给 Layer 3。Layer 3 只在「尚未全部达标」时做 Expert / PPO / CLS，最后由反思给出 `done / tune_params / modify_law`，反馈回下一轮。

LAYER 1 · 交互与主循环

cli_agent.py 解析自然语言任务与 T/G/AP/R 工况，初始化 RAG / 经验库 / Hermes / 优化工作流，驱动外层迭代，处理中断菜单与最终报告。 规划器 `IntelligentTaskPlanner` 在进入 Hermes 之前选定执行模式。

▼ 脚本路径 + 工况 + 上一轮建议

LAYER 2 · Hermes 纯编排器

不自己做 RL。 两种动作：直接调工具，或 `call_specialist(agent_type, task, context)` 委托子智能体。子智能体靠**工具白名单隔离**，防止递归和串工具。MODIFY_LAW 推荐链：检索 → 生成/校验 → 仿真 → 判断。

rag_agent	matlab_agent	sim_agent	analysis_agent	memory_agent
知识检索	生成 / 语法 / 适配性	跑蒙特卡洛	判断与经验库	长期记忆读写

▼ script_path + 初始五项指标

LAYER 3 · 优化工作流（async）

OptimizationWorkflow：若 Layer 2 已全部达标，可跳过 RL，只做终检与回写。 否则走 Expert 调参 → PPO（10 维）→ 约束局部搜索 CLS → 单指标收尾。 `JudgmentAgent` 在新最优出现时做规则验收；`ReflectionAgent` 做路径决策，不替代数值门。

和「多智能体」常见想象的差别

- **有中心，但中心很薄。** Hermes 是调度器，不是领域专家，也不掌握仿真真值。
- **有记忆，但记忆分库。** RAG 存知识，ParameterExperience 存参数，agent_memory 存情节，互不替代。
- **有辩论空间，但验收不投票。** 五项指标用规则 AND，任一 NG 即未完成；LLM 不能把 4/5 说成通过。
- **有循环，但预算封顶。** 外层 ≤5，等预算对照上限 75 候选 / 225 次开发集 MC。

3. 角色、数据协议与一次任务怎么走完

角色	实现	输入	输出 / 权限边界
主循环	`cli_agent.py`	prompt、config、消融开关	何时开下一轮；不直接改参数
任务规划	`IntelligentTaskPlanner`	自然语言 + 工况参考	REUSE / TUNE / MODIFY；可拆子任务
编排器	Hermes `AIAgent`	规划结果 + 上一轮建议	工具调用或委托；看不到 PPO 内部状态
五类专家	`SpecializedSubAgent`	结构化 `AgentMessage`	强制 JSON 块；只能看见本角色工具
判断	`JudgmentAgent`	仿真指标 + 任务要求	satisfied 仅当五项全 OK；规则优先
反思	`ReflectionAgent`	指标、历史、脚本片段	needs_optimization、next_action、建议文本
优化后端	`optimizer_registry`	同一仿真评估器	可换 hybrid_ppo / direct_ppo / CMA / random

角色	实现	输入	输出 / 权限边界
物理真值	MATLAB / Octave	`monte_carlo_single.m` + RL_PARAMS	五项指标；不被 LLM 改写

子智能体之间怎么传数据

不用共享黑板乱写。编排器用 `context` 把上一步 JSON 传给下一步。每个子智能体必须在回复末尾给出 ```` ```json ... ``` ````，Hermes 从中抽取 `script_path`、`satisfied` 等字段。 这是为了让「MATLAB 路径、仿真数字、是否达标」成为机器可读契约，而不是一段散文。

```
任务 prompt
  → Planner 选定 TUNE_PARAMS / MODIFY_LAW / REUSE_HISTORY
  → Hermes: rag_agent → matlab_agent → sim_agent → analysis_agent
  → 若五项已过: Layer3 只终检 + 经验回写
  → 若未过: Expert / PPO / CLS (判断可早停)
  → Reflection: done 结束; 否则 next_action 回到 Layer1
  → 达 max_iterations 或预算耗尽则强制收工, 输出当前最优
```

三种设计路径，对应三种「智能体参与深度」

REUSE_HISTORY

历史参数已够用。Layer 2 几乎不改算法，Layer 3 可以很轻。多智能体成本最低。

TUNE_PARAMS

结构不动，只搜 10 维参数。这是论文主路径：LLM 诊断瓶颈，数值优化器负责搜索。

MODIFY_LAW

要改 `gf()` 数学结构。必须走检索—生成—语法—适配性—仿真。T4 等高过载工况才更常触发。

4. 为什么使用这套架构：六条设计原则

1. 编排与优化解耦

RL 不进 Hermes 工具集。否则一次超长蒙特卡洛会卡在编排器超时里，而且消融「去 RL」会拆不动编排层。 论文要证明的是「闭环设计框架」，不是「把 PPO 塞进聊天工具」。

2. 工具隔离，而不是人设隔离

子智能体不是再开五个不受控的大模型会话，而是同一套 Hermes 上限制可见工具。 检索智能体调不到 `run_simulation`，仿真智能体也改不了知识库。这降低串工具、递归委托和上下文污染。

3. 规则验收，LLM 只解释

制导指标是硬门：命中率、SEP、过载、相位裕度、带宽。判断智能体先做数值 AND，通过才考虑早停。 反思负责「下一步改结构还是改参数」，不能把未达标说成达标。

4. 快路径与完整路径并存

T4 可跳过 Hermes 直接 generate→语法→仿真→判断；Layer 2 已满足则可跳过 PPO。 架构允许「贵的智能体只在高不确定性时出场」，这是工程可行性，不是理论上的纯粹多智能体。

5. 消融必须能拔插件

`config.yaml` 的 ablation 开关对应适配性、语法、RL、经验库、记忆、反思、外层轮次。 如果所有能力糊在一个 Agent 的 prompt 里，第 4.2 节消融在方法上就不成立。

6. 同一评估器，可换搜索算法

Layer 3 后端走注册表：hybrid PPO、Direct PPO、CMA-ES、随机搜索共用仿真评估接口。 这让「多智能体框架」和「优化器对照」成为同一套尺子上的不同插件，而不是两套程序。

5. 与其他架构相比，为什么更适合本项目

对照对象按「本仓库已经跑过的」和「领域里常见的」分开。单智能体 Hermes（SAH）、Direct PPO、CMA-ES、随机搜索 都已作为等预算基线存在；下面同时解释为什么主系统不采用那些更「纯」的多智能体风格。

架构	它擅长什么	用在本项目会怎样	适合度
单智能体 ReAct （本仓库 SAH）	实现快，工具全在一个上下文里	能调参、能仿真，但去掉了分层委托、反思诊断和 Hermes 记忆注入。n10 对齐后开发集/测试集 12/12，说明**搜索本身很强**；但它无法回答「哪一层贡献了什么」，也无法安全地改制导律结构。主系统要的是可解释闭环，不是单纯最高可行率。	**基线合适** **主系统不够**
纯数值优化 Random / CMA / PPO	连续参数空间、预算清晰	不会写 MATLAB、不会选 MODIFY_LAW、不会读知识库。适合 Table 5 对照，不能单独承担「智能设计」。	**对照层合适**
扁平辩论 AutoGen / 圆桌投票	开放问题、方案发散	仿真昂贵，辩论轮次会把 MC 预算打爆；指标真值会被「说得通」的文本稀释。制导约束不是多数表决问题。	**不适合主环**
纯 Supervisor–Worker （LangGraph 风格）	状态机清晰、可恢复	思想上接近本项目。差别是：本项目把 Worker 做成**工具子集**而不是独立图节点，并把最贵的 PPO 放在编排图外面。若整段 PPO 也进图，超时与消融都会变难。	**近邻，** **但本项目更省**
无反思流水线 DAG	一次生成、一次仿真、结束	无法处理「结构错了还是参数错了」。T4 过载超限需要在 TUNE 与 MODIFY 之间切换。没有外层闭环就没有设计路径。	**只适合演示**
全 LLM 调参 （让模型直接给 w1…）	少写优化器	10 维、带物理边界、要上百次 MC。LLM 采样效率远低于 PPO/CMA，且不可复现随机种子协议。	**不适合搜索层**
本项目：分层编排闭环	异构任务 + 硬约束 + 可消融 + 可换优化器	自然语言走规划与生成，仿真走 MATLAB，搜索走注册表，验收走规则。每一层都可以单独关掉做实验。	**主系统**

> 适合度的判据不是「智能体数量最多」，而是三件事同时成立：**(1) 仿真真值不被语言模型覆盖；(2) 结构修改与参数搜索可分开消融；(3) 后面加新专家/新优化器时不必重写主循环。**

6. 当前使用层：这套架构已经在支撑什么

6.1 主系统工作流

- 论文主方法 GSMultiAgent：Layer1 解析 → Layer2 编排/生成/仿真 → Layer3 优化与反思。
- 外层上限 5 轮；记忆复用与约束满足后可以早停，因此表上 GSM 的 MC 常是 1 / 7 / 22，而不是搜满 225。
- 设计路径被记录为 `first_pass` / `TUNE_PARAMS` / `MODIFY_LAW`，用于解释「为什么有的格子首轮就停、有的还继续」。

6.2 科学实验已经利用的模块边界

组件消融（第 4.2 节）

去适配性、去语法、去 RL、去经验库、去记忆、单轮迭代。若没有分层，这些开关只能变成「改 prompt」，审稿无法复现。

优化器对照（第 4.3 节）

同一 225 次 MC 上限下比较 Random、CMA-ES、Direct PPO、单智能体 Hermes 与完整 GSM。架构把「框架」和「搜索算法」拆开，对照才合法。

校正管线

语法检查、适配性证明、T4 快路径、BW 收尾。这些是 Layer2/Layer3 的可插拔策略，不是新开一套 Agent 框架。

人机协同已预留

Ctrl+C 菜单可改指令、晋升记忆、继续或退出。说明 Layer1 被有意做成「可打断的实验驾驶舱」，而不只是批处理脚本。

6.3 当前架构已经暴露出的使用纪律

- 表上 GSM / SAH 的 `first_success` 是**最终开发集可行**，不是「第一轮成功就停」。
- 基线第一次可行只是 JSON 历史里的书签；为了等预算，它们会继续搜满。
- LLM 顾问默认倾向继续优化。这是能力，也是成本来源：架构允许早停，但不把「第一次可行」当成唯一目标。

7. 扩展层：以后往哪长，而不必推翻骨架

扩展应沿着现有缝隙加插件。下面按「改动半径从小到大」排列。

扩展方向	挂载点	为什么现有架构接得住	注意
新工况 / 新指标	任务要求解析 + `JudgmentAgent` 规则 + MATLAB 输出字段	判断合同已经是「任务里写到的指标必须全过」。加 G 类几何、AP 退化，不必改 Hermes。	先改仿真契约，再改规则，最后才动 LLM 提示。
新优化算法	`optimizer_registry`	已有 hybrid_ppo / direct_ppo / cma_es / random_search。贝叶斯优化、差分进化可按同一评估器接入。	必须遵守同一候选预算，否则又和 GSM 不对齐。
新子智能体	`SUB_AGENT_CONFIGS` + 工具白名单	例如气动辨识、弹目几何检查、报告撰写。只要输出强制 JSON，就能插入 MODIFY_LAW 链。	不要给子智能体注册对方的工具名，避免全局注册表冲突。
新知识源	`knowledge_base/` + RAG 工具	检索智能体与生成智能体已解耦。换文档不必换优化器。	保持「检索结果进 context、不直接改指标」。
更强的结构搜索	MODIFY_LAW + 适配性证明	结构修改已从调参路径分出。可加程序综合、模板库枚举，仍由仿真验收。	结构搜索极贵，必须有独立预算，不能混进 225 次参数 MC。
多人/多机	Layer1 任务队列 + 经验库	PE 与 RAG 已是外部存储。Windows / Ubuntu 双机实验已经按目录同步结果。	不要让两台机器写同一份未加锁经验库。

扩展方向	挂载点	为什么现有架构接得住	注意
在线/硬件在环	把 `sim_agent` 的仿真后端换成 HIL 接口	Layer 2 只依赖「跑完返回五项指标」的工具合同。	时序与安全互锁必须放在仿真适配器，不要放进 LLM。
换成别的编排内核	`hermes_integration.py` 适配层	主循环并不绑定 Hermes 的对话格式；缺 Hermes 时 CLI 仍可完成 Step 1。理论上可换 LangGraph，但要重做工具隔离。	值得换的前提是：超时、子智能体隔离、JSON 契约能等价实现。

扩展时建议遵守的分层纪律

1. **真值向下沉。** 新指标先出现在 MATLAB，再进入判断规则，最后才进入反思提示词。
2. **搜索可替换。** 新算法进注册表，不进 Hermes 工具列表。
3. **专家可增加。** 新专家只拿自己的工具子集，用 JSON 与上下游衔接。
4. **预算单列。** 结构搜索、网格扫描、硬件试验不要偷偷占用论文里的 225 次开发集 MC。
5. **消融先行。** 新模块必须能在 `config.yaml` 关掉，否则无法写进下一轮实验。

8. 这套架构的边界：什么时候不该再加 Agent

已经足够模块化的部分

- 参数搜索：用注册表，不要再包一层「优化讨论组」。
- 指标验收：继续规则 AND，不要改成投票。
- 等预算对照：保持同一评估器，避免每个 Agent 自己数 MC。

继续加 Agent 只会变差的情况

- 为了「看起来更智能」让多个模型复述同一组仿真数字。
- 把 PPO 每一步都交给 LLM 批准，墙钟和随机性都会失控。
- 让子智能体互相递归 `call_specialist`，工具隔离就被打穿。

GSMultiAgent 的多智能体，是面向制导设计工作流的分层控制系统，不是通用聊天应用。 它适合本项目，是因为项目的困难在于「语言、代码、物理仿真、数值搜索」必须同时正确； 分层之后，每一层都可以被实验证明、被替换、被扩展。

依据仓库当前实现整理：`cli_agent.py`、`multi_agent/integration/`、`optimization_workflow.py`、`optimizer_registry.py`、`config.yaml`、`README.md`。 文件位置 `docs/multi_agent_architecture.html`。